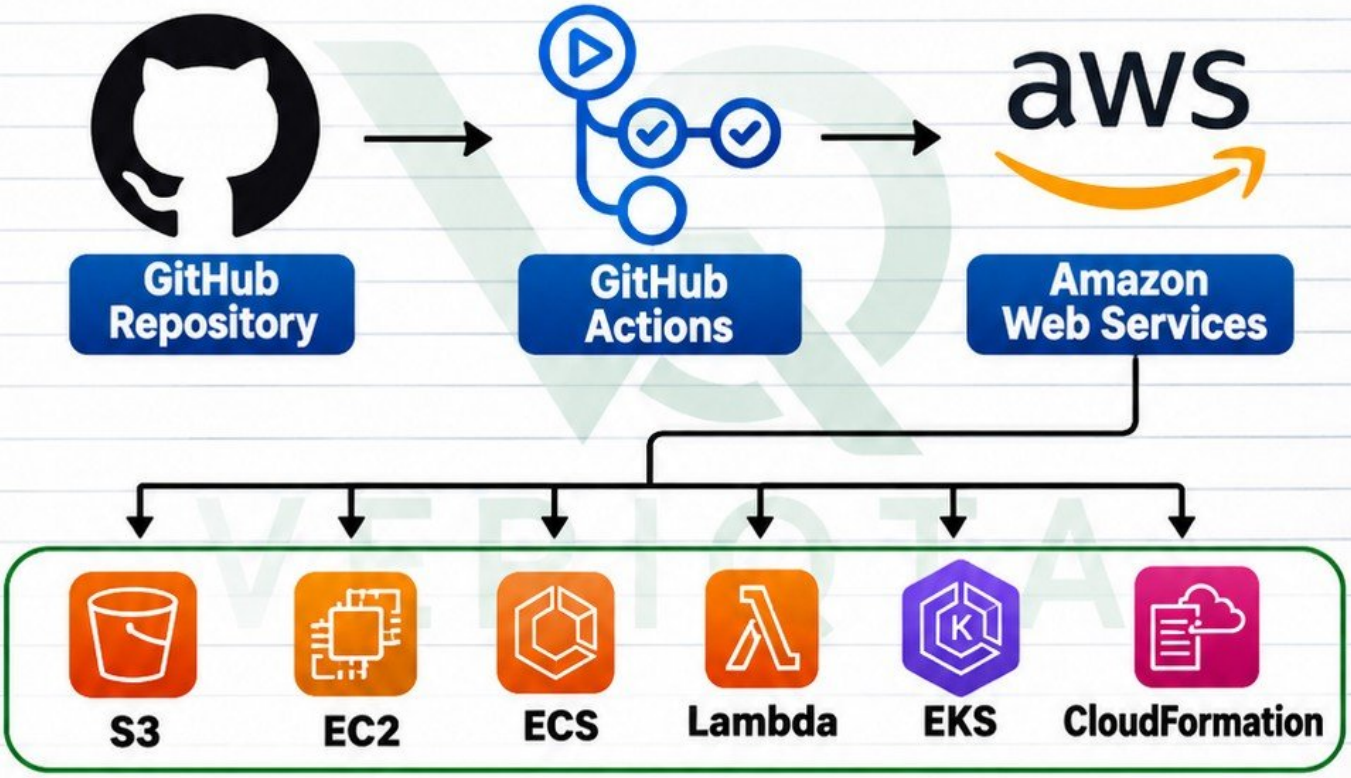
VERIQTA

GitHub Actions for AWS:

20 Real Deployment Workflows



- ✅ **Build Automated CI/CD Pipelines**
- ✅ **Deploy Applications to AWS**
- ✅ **Work with EC2, ECS, EKS, Lambda and S3**
- ✅ **Automate Infrastructure with Terraform & CloudFormation**
- ✅ **Implement Security, Monitoring and Rollback**

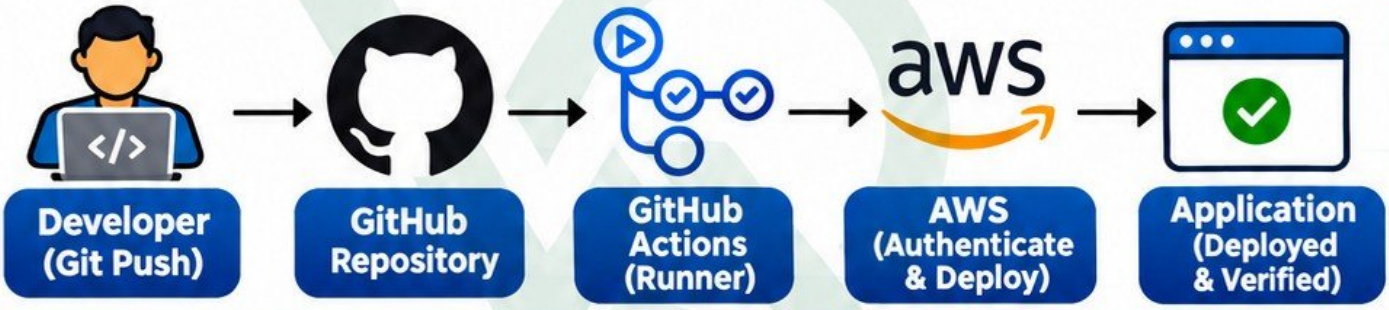
@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

GitHub Actions to AWS: Deployment Architecture

What You Will Learn

- ✅ How a Git push triggers a GitHub Actions workflow.
- ✅ The path from repository to runner, AWS authentication, deployment, and verification.
- ✅ Key terms: workflow YAML, jobs, steps, runners, triggers, and permissions.

Deployment Architecture



Key Terms You Must Know

- **Workflow YAML:** A file (.yml) that defines the automation.
- **Job:** A unit of work (e.g., build, test, deploy).
- **Step:** A single task in a job (e.g., run a command, build, deploy).
- **Runner:** The machine that executes your workflow (GitHub-hosted or self-hosted).
- **Trigger:** The event that starts the workflow (e.g., push, pull request).
- **Permissions:** The access the workflow has (e.g., to deploy to AWS).

Real-World Example

A developer merges code to the main branch. GitHub Actions automatically:

1. Builds and tests the application.
2. Authenticates to AWS using a secure role (OIDC).
3. Deploys the new version (e.g., to EC2, ECS, or Lambda).
4. Verifies the deployment (health check or status check).

Typical Workflow YAML (Snippet)

```yaml
name: Deploy to AWS
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
```

Key Takeaways

- ✅ A Git push can automatically build, test, and deploy to AWS.
- ✅ The workflow runs on a runner using a YAML file.
- ✅ Use secure authentication (OIDC) for AWS access.
- ✅ Always verify the deployment after it completes.

VERIQTA

AWS Authentication Without Access Keys: GitHub OIDC

What is OIDC?

- OpenID Connect (OIDC) is a secure way for GitHub Actions to get temporary AWS credentials.
- It removes the need for long-lived AWS access keys.
- GitHub proves its identity to AWS, and AWS gives short-lived, limited-permission access.

Why Use It?

- More secure (no stored access keys).
- Uses short-lived credentials (automatic expiry).
- Tighter control with IAM role policies.
- Restrict access to a specific repository, branch, or environment.

How It Works



IAM Trust Policy (Example)

```
{
  "Effect": "Allow",
  "Principal": {
    "Federated":
      "arn:aws:iam::<ACCOUNT_ID>:
      oidc-provider/token.actions.githubusercutent.com"
  },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com.aud":
        "sts.amazonaws.com"
    },
    "StringLike": {
      "token.actions.githubusercontent.com:sub":
        "repo:veriqta-app/*:ref:refs/heads/main"
    }
}
```

GitHub Actions (Usage)

```
- name: Configure AWS Credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::<ACCOUNT_ID>:
          role/GitHubActionsDeployRole
    aws-region: us-east-1
```

Verify the Identity

```
- name: Verify AWS identity
  run: aws sts get-caller-identity
```

Expected Output

```
{
  "Account": "123456789012",
  "Arn": "arn:aws:sts::123456789012:assumed-role/
     GitHubActionsDeployRole/GitHubActions",
  "UserId": "AROAXXXXX:GitHubActions"
}
```

Common Failure

⚠ **AccessDenied:** Not authorized to perform sts:AssumeRoleWithWebIdentity.

How to Fix It

- ✓ Check the role trust policy conditions.
- ✓ Ensure the repository and branch match.
- ✓ Confirm the OIDC provider is correctly configured.
- ✓ Verify the IAM role has the required permissions (e.g., deploy to S3, ECS, or Lambda).

Deploy a Static Website to Amazon S3

What You Will Do

- Create a private S3 bucket for website assets.
- Build a simple frontend with GitHub Actions.
- Upload build artifacts using aws s3 sync.
- Verify the uploaded objects and deployment.
- Understand the risk of --delete.

Deployment Architecture



GitHub (Push Code) → GitHub Actions (Build & Sync) → Amazon S3 (Static Website Files)

Workflow Steps

1. Push code to GitHub (e.g., main branch).
2. GitHub Actions checks out the code.
3. Install dependencies and build the frontend (e.g., npm run build).
4. Sync the build output to an S3 bucket using aws s3 sync.
5. Verify the files and website result on S3 (or via CloudFront).

Key Commands

aws s3 sync ./dist s3://veriqta-site/

- Uploads the build files to the S3 bucket.

aws s3 sync ./dist s3://veriqta-site/ --delete

- Uploads files and removes files in the bucket that are no longer in ./dist.
- Use with care in production.

About --delete



The --delete flag removes files from S3 that are not in your current build. It can delete useful files if your build is incomplete.

Example: GitHub Actions (Snippet)

```yaml
name: Deploy to S3
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci && npm run build
      - run: aws s3 sync ./dist s3://veriqta-site/
```

Verify the Deployment

aws s3 ls s3://veriqta-site/

- Lists the uploaded files in the bucket.

aws s3 website s3://veriqta-site/

- Shows the website configuration.

Production Connection



S3 (Origin) → CloudFront (CDN) → Your Website (Fast & Global)

- S3 hosts your static files, and CloudFront delivers them globally with caching.

Key Takeaway Use GitHub Actions to build and sync your site to S3, then use CloudFront for a fast, secure, and production-ready website.

Deploy a Frontend Through CloudFront

What You Will Do

- Connect CloudFront to a private S3 origin using Origin Access Control (OAC).
- Deploy updated frontend assets with GitHub Actions.
- Use hashed filenames and cache headers.
- Invalidate selected CloudFront paths when needed.
- Diagnose stale content, missing assets, and HTTP 403 errors.

Architecture



User (Browser) → CloudFront (CDN) → S3 (Private Origin) with OAC

Deployment Flow

1. Build the frontend (e.g., npm run build).
2. Sync build files to the S3 bucket (aws s3 sync).
3. Invalidate CloudFront paths (to clear old cached files).
4. Verify the site loads with the latest version.

Key AWS Concepts

- **CloudFront:** A global CDN that delivers your website with low latency.
- **Origin Access Control (OAC):** A secure way for CloudFront to access a private S3 bucket.
- **Hashed filenames:** File names with a unique hash (e.g., app.8f3d2e.js) so new versions are detected.
- **Cache headers:** Tell CloudFront how long to cache files (e.g., Cache-Control: max-age=31536000).

GitHub Actions (Key Steps)

```
name: Deploy Frontend to S3 + CloudFront
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build
      - run: aws s3 sync ./dist s3://veriqta-site/
      - run: aws cloudfront create-invalidation \
          --distribution-id $CF_DIST_ID \
          --paths "/index.html" "/assets/*"
```

Common Issues & Fixes

Issue	Possible Cause & Fix
Stale content (old version)	■ Files are cached by CloudFront. ■ Fix: Invalidate the required paths.
Missing assets (404)	■ Incorrect file path or build output. ■ Fix: Verify the dist/ folder and S3 sync command.
HTTP 403 (Access Denied)	■ S3 bucket not properly configured for OAC. ■ Fix: Confirm the OAC and S3 bucket policy.

Example Cache Headers

```
# Long cache for static files (JS, CSS, images)
Cache-Control: public, max-age=31536000

# Short cache for HTML (so updates show quickly)
Cache-Control: public, max-age=300
```

Production Lesson

Cache strategy matters as much as deployment success. Plan what to cache (long) and what to refresh quickly (short).

✓ Key Takeaway

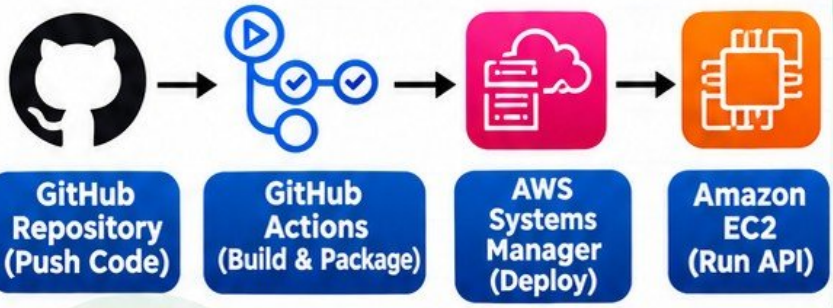
Use GitHub Actions to build and deploy to S3, connect it to CloudFront with OAC, set proper cache headers, and invalidate only the paths you need.

Deploy a Node.js API to Amazon EC2

What You Will Do

- Build and test a Node.js application in GitHub Actions.
- Package a versioned deployment artifact.
- Deliver the artifact to EC2 using AWS Systems Manager.
- Restart a systemd-managed application.
- Verify the service with systemctl, journalctl, and a health endpoint.

Deployment Architecture



Key Concepts

- **GitHub Actions:** Automates build, test, and deployment.
- **AWS Systems Manager (SSM):** Securely runs commands on EC2 (without SSH).
- **Systemd:** Manages your Node.js application as a service.
- **Deployment artifact:** A versioned package (e.g., app-v1.0.0.tar.gz).

Deployment Steps

1. Push code to GitHub (main branch).
2. Run tests and build the Node.js app (e.g., npm ci && npm test && npm run build).
3. Package the app (e.g., tar -czf app-v1.0.0.tar.gz dist/).
4. Send the artifact to EC2 using SSM (aws ssm send-comand).
5. Restart the application with systemctl.
6. Verify the service and health endpoint.

GitHub Actions (Key Snippet)

```yaml
name: Deploy Node.js API to EC2
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm test
      - run: tar -czf app-${{ github.sha }}.tar.gz dist/
      - run: |
          aws ssm send-command \
          --instance-ids i-0abc123def456 \
          --document-name "AWS-RunShellScript" \
          --parameters commands=["/opt/deploy.sh"]
```

Systemd Service (Node.js)

```
[Unit]
Description=Node.js API
After=network.target

[Service]
WorkingDirectory=/opt/app
ExecStart=/usr/bin/node dist/server.js
Restart=always

[Install]
WantedBy=multi-user.target
```

Verify the Deployment

`systemctl status api`	– Check if the service is running.
`journalctl -u api -f`	– View real-time application logs.
`curl http://<EC2-IP>:3000/health`	– Test the API health endpoint.

Common Failure

Deployment succeeds, but the application fails to start.

Common reasons:

- Wrong working directory or file path.
- Missing dependencies (npm ci not run).
- Port already in use.
- Environment variables not set.

Production Tips

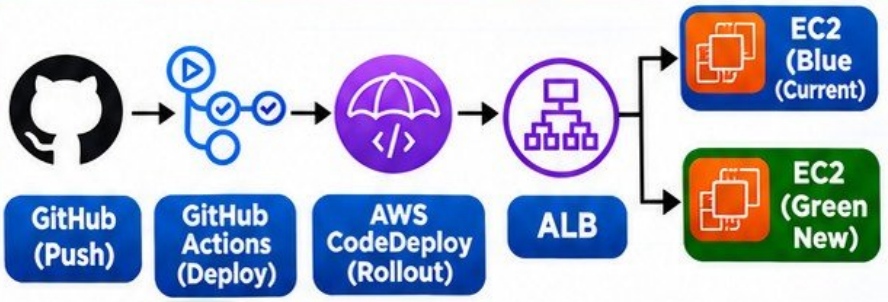
- Use SSM instead of SSH for secure, audited deployments.
- Run the app with systemd for automatic restart and reliability.
- Always verify the service and health endpoint after deployment.

Zero-Downtime Deployment to EC2 with CodeDeploy

What You Will Do

- Understand in-place versus blue/green deployments.
- Configure AWS CodeDeploy with an Application Load Balancer (ALB).
- Create an appspec.yml file and deployment lifecycle hooks.
- Automate deployment from GitHub Actions.
- Verify target health before completing the deployment.

Architecture (Blue/Green with CodeDeploy)



Deployment Strategies

In-Place

- Update the application on the same EC2 instances.
- Shorter setup, small risk of brief disruption.

Blue/Green

- Deploy a new version to a separate set of instances (Green), test, then switch traffic.
- Zero downtime and safer releases.

CodeDeploy Lifecycle Hooks

Hook	Purpose
BeforeInstall	Run tasks before new files are copied (e.g., stop service, backup).
AfterInstall	Run setup after files are copied (e.g., install dependencies).
ApplicationStart	Start the application (e.g., systemctl start).
ValidateService	Verify the app is healthy (e.g., HTTP check); fail the deployment if not healthy.

appspec.yml (Example)

```
version: 0.0
ios:
  os: linux
files:
  - source: /
    destination: /opt/app
hooks:
  BeforeInstall:
    - location: scripts/stop.sh
      timeout: 300
  AfterInstall:
    - location: scripts/install.sh
      timeout: 300
  ApplicationStart:
    - location: scripts/start.sh
      timeout: 300
  ValidateService:
    - location: scripts/health-check.sh
      timeout: 300
```

GitHub Actions (Deployment Snippet)

```
name: Deploy with CodeDeploy
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Create deployment
      run: |
        aws deploy create-deployment \
        --application-name veriqta-api \
        --deployment-group-name prod-ec2 \
        --s3-location bucket=veriqta-artifacts,\\
          key=app-v1.0.0.zip,bundleType=zip
```

Verify the Deployment

`aws deploy get-deployment \ --deployment-id <id>`	Check deployment status.
`aws elbv2 describe-target-health \ --target-group-arn <arn>`	Confirm EC2 targets are healthy.
`curl http://<alb-dns>/health`	Test the application endpoint.

Common Issues & Fixes

Issue	Fix
Deployment fails at hook	Check the hook script logs (/opt/codedeploy-agent/deployment-root/).
Targets unhealthy	Verify the app is running and listening on the correct port.
ALB returns 5xx	Check security groups, health check path, and application logs.

Production Lesson

A successful file transfer is not a successful application release. Always validate the application (health, logs, and target health) before completing the deployment.

Build and Push Docker Images to Amazon ECR

What You Will Do

- Create an Amazon Elastic Container Registry (ECR) repository.
- Build a Docker image in GitHub Actions.
- Authenticate securely to ECR.
- Tag images with a Git commit SHA instead of using latest.
- Push the image and inspect its immutable digest.

Deployment Flow



GitHub Actions (Build & Push) → Docker (Build Image & Tag) → Amazon ECR (Private Image Registry)

Workflow Steps

1. Create an ECR repository (in AWS).
2. Build the Docker image in GitHub Actions.
3. Authenticate to ECR (AWS CLI).
4. Tag the image with the commit SHA.
5. Push the image to ECR.
6. Verify the image and its digest in ECR.

Key Commands

```
$ aws ecr create-repository
  --repository-name veriqta-api
```

Create an ECR repository.

```
$ aws ecr get-login-password |
  docker login --username AWS --password-stdin
  <ACCOUNT_ID>.dkr.ecr.<region>.amazonaws.com
```

Authenticate Docker to ECR.

```
$ docker push <ACCOUNT_ID>.dkr.ecr.<region>.
  amazonaws.com/veriqta-api:<tag>
```

Push the tagged image to ECR.

GitHub Actions (Workflow Snippet)

```
name: Build and Push to ECR
on:
  push:
    branches: [ main ]
jobs:
  build-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::<ACCOUNT_ID>:role/GitHubECRRole
          aws-region: us-east-1
      - run: |
          IMAGE_TAG=${{ github.sha }}
          docker build -t veriqta-api:$IMAGE_TAG .
          aws ecr get-login-password --region us-east-1 | \
            docker login --username AWS --password-stdin \
            <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com
          docker tag veriqta-api:$IMAGE_TAG \
            <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/veriqta-api:$IMAGE_TAG
          docker push <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/veriqta-api:$IMAGE_TAG
```

Image Tagging (Best Practice)

✗ veriqta-api:latest	Avoid: changes frequently, not traceable.
✓ veriqta-api:3f2a9e7	Use: Git commit SHA (immutable and traceable).

Verify the Image (Digest)

```
$ aws ecr describe-images \
    --repository-name veriqta-api \
    --query 'imageDetails[0].imageDigest'

"sha256:9b7e2c0d7e1a4f5e2e9b..."
```

Common Issues & Fixes

Issue	Likely Cause	Fix
AccessDeniedException	IAM role lacks ECR permissions.	Add ecr:GetAuthorizationToken, ecr:PutImage, ecr:InitiateLayerUpload, ecr:UploadLayerPart, ecr:CompleteLayerUpload.
no matching manifest (architecture)	Image built for the wrong architecture (e.g., arm64 vs amd64).	Build for the correct platform: docker build --platform linux/amd64 ...
image not found / wrong tag	Tag not created or pushed correctly.	Verify the tag: docker images and ensure the same tag is pushed to ECR.

✓ Key Takeaway — Use GitHub Actions to build, tag (with commit SHA), and push Docker images to ECR, and always verify the image digest after the push.

Deploy a Containerized API to Amazon ECS Fargate

What You Will Do

- Create an ECS task definition, service, and Application Load Balancer (ALB).
- Build and publish a container image to Amazon ECR.
- Update the task definition with the new image.
- Deploy using GitHub Actions and wait for service stability.
- Verify target group health and application responses.

Architecture: GitHub Actions → ECS Fargate



GitHub Actions (Build & Push) → Amazon ECR (Container Image) → Amazon ECS Fargate (Run API) → Application Load Balancer (ALB) → API (HTTP/HTTPS)

Key AWS Components

ECS Fargate	Runs your container without managing servers.
ECR	Stores your container image securely.
ALB	Distributes traffic to healthy containers.
Task Definition	Defines the container, CPU, memory, port, and settings.

Deployment Steps

1. Build and push a Docker image to ECR.
2. Create or update the ECS task definition (with the new image).
3. Create an ECS service (Fargate) and connect it to an ALB (target group).
4. Deploy from GitHub Actions and wait for service to reach a stable state.
5. Verify target group health and test the API endpoint (e.g., curl or browser).

GitHub Actions (Workflow Snippet)

```
name: Deploy API to ECS Fargate
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - uses: aws-actions/configure-aws-credentials@v4
    - run: docker build -t veriqta-api:$GITHUB_SHA .
    - run: aws ecr get-login-password | \
        docker login --username AWS --password-stdin \
        $ECR_URI
    - run: docker push $ECR_URI:$GITHUB_SHA
    - run: aws ecs update-service \
        --cluster veriqta-cluster \
        --service veriqta-api \
        --force-new-deployment
```

Verify the Deployment

`aws ecs describe-services \ --cluster veriqta-cluster \ --services veriqta-api`	Check that the service is ACTIVE and STABLE.
`aws elbv2 describe-target-health \ --target-group-arn <tg-arn>`	Confirm targets are HEALTHY.
`curl http://<ALB-DNS>/health`	Test the API response (should return 200 OK).

Common Failure & Fixes

Issue	Likely Cause	Fix
Tasks repeatedly stop (Exit 1)	Wrong port, missing config, or app crash.	Use the correct container port and check app logs.
Target UNHEALTHY	Health check path incorrect or app not ready.	Use a valid health check path (e.g., /health) and allow enough startup time.
Image pull failed	ECR permissions missing.	Ensure the ECS task role can pull from ECR (ecr:GetAuthorizationToken, ecr:BatchGetImage, ecr:GetDownloadUrlForLayer).

✓ Production Lesson

A successful deployment means a healthy, serving API. Always verify target health and application response before considering the release complete.

Deploy an AWS Lambda Function Through GitHub Actions

What You Will Do

- Package a Python Lambda function and its dependencies.
- Configure IAM execution permissions.
- Deploy a versioned function using AWS CLI or a framework.
- Publish a Lambda version and update an alias.
- Verify invocations and CloudWatch logs.
- Troubleshoot missing dependencies, timeouts, and runtime errors.

Deployment Architecture



GitHub Repository (Code) → GitHub Actions (Build & Deploy) → AWS Lambda (Function + Version + Alias) → Amazon CloudWatch (Logs & Monitoring)

Key Concepts

AWS Lambda	Runs your code without managing servers.
IAM Role	Gives the function permission to access AWS resources.
Lambda Version	A numbered, immutable release of your function.
Alias	A stable name (e.g., prod) that points to a version.
CloudWatch	Logs, metrics, and errors for your function.

Deployment Steps

1. Write the Lambda function (Python) and add a requirements.txt.
2. Package the code and dependencies (zip).
3. Configure an IAM execution role (e.g., AWSLambdaBasicExecutionRole).
4. Deploy or update the function from GitHub Actions (AWS CLI or SAM).
5. Publish a new version and update an alias (e.g., prod).
6. Verify the function with a test invocation and check CloudWatch logs.

GitHub Actions (Workflow Snippet)

```yaml
name: Deploy Lambda (Python)
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - name: Package function
      run: |
        pip install -r requirements.txt -t package/
        cp -r src/* package/
        cd package && zip -r ../lambda.zip .
    - name: Deploy and publish
      run: |
        aws lambda update-function-code \
          --function-name veriqta-api \
          --zip-file fileb://lambda.zip
        aws lambda publish-version --function-name veriqta-api
        aws lambda update-alias --function-name veriqta-api \
          --name prod --function-version ${{ steps.version.outputs.version }}
```

Verify the Deployment

`aws lambda get-function \ --function-name veriqta-api`	Check the function configuration and latest version.
`aws lambda list-aliases \ --function-name veriqta-api`	Confirm the alias (e.g., prod) points to the new version.
`aws lambda invoke \ --function-name veriqta-api \ --payload '{}' response.json`	Test the function and review the output (response.json).
`aws logs tail \ /aws/lambda/veriqta-api \ --follow`	View real-time logs in CloudWatch.

Common Failure & Fixes

Issue	Likely Cause	Fix
ModuleNotFoundError	Dependencies not packaged.	Install with -t and zip with the code.
Timeout	Function takes longer than the limit.	Increase the timeout or optimize the code.
AccessDenied	IAM role missing required permissions.	Attach the correct policy (e.g., CloudWatch, S3, etc.).
Runtime error (Exit 1)	Code or environment issue (e.g., wrong Python version).	Check CloudWatch logs and ensure the correct Python runtime is configured.

✓ Production Lesson — Deployment is not complete until the new Lambda version is published, the alias is updated, and the function is verified.

Deploy a Serverless API with API Gateway and Lambda

What You Will Do

- Define API Gateway routes and Lambda integrations.
- Manage the application using AWS SAM templates.
- Run build and template validation in GitHub Actions.
- Deploy infrastructure and application code together.
- Test endpoints using curl.
- Troubleshoot HTTP 500 errors caused by integration issues or missing permissions.

Architecture: Serverless API



GitHub Repository (Push Code) → GitHub Actions (Build & Deploy) → AWS SAM (Build & Package) → Amazon API Gateway (HTTP API) → AWS Lambda (Handle Request)

Key Components

AWS Lambda	Runs your backend code on demand.
API Gateway	Exposes HTTP endpoints and routes requests to Lambda.
AWS SAM	A framework to define, build, and deploy serverless apps (CloudFormation under the hood).
IAM Role	Permissions for Lambda to access AWS resources (e.g., CloudWatch, S3, DynamoDB).

Deployment Steps

1. Create the SAM project (template.yaml) with API Gateway and Lambda.
2. Define API routes and Lambda integration in the SAM template.
3. Run build and validate in GitHub Actions (sam build and sam validate).
4. Deploy using SAM (sam deploy) from GitHub Actions.
5. Test the API endpoints using curl.

SAM Template (template.yaml – Excerpt)

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Resources:
  ApiFunction:
    Type: AWS::Serverless::Function
    Properties:
      Runtime: python3.12
      Handler: app.lambda_handler
      Events:
        Api:
          Type:Api
          Properties:
            Path: /hello
            Method: GET
```

GitHub Actions (Deploy with SAM)

```
name: Deploy Serverless API
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - uses: aws-actions/setup-sam@v2
    - name: Build and validate
      run: |
        sam build
        sam validate
    - name: Deploy
      run: |
        sam deploy --no-confirm-changeset \
        --stack-name veriqta-serverless-api
```

Test the API

```
$ curl -s https://<api-id>.execute-api.<region>.amazonaws.com/hello
```

Expected response:

```
{"message": "Hello from Lambda"}
```

Verify Deployment

- SAM stack created/updated successfully.
- API Gateway route is active.
- Lambda invocations and logs are available in CloudWatch.

Common Failure & Fixes

Issue	Likely Cause	Fix
HTTP 500 (Internal Server Error)	Integration error, code runtime error, or missing permissions.	Check CloudWatch logs, fix the code, and ensure the Lambda role has the required permissions.
AccessDenied	Lambda role lacks permission to access a needed AWS service.	Update the IAM role policy (e.g., logs, DynamoDB, S3).
Route not found (404)	Incorrect or missing route in the SAM template.	Verify the Path and Method in template.yaml and redeploy.
Invalid template	SAM template syntax or resource configuration error.	Run sam validate and fix the reported errors.

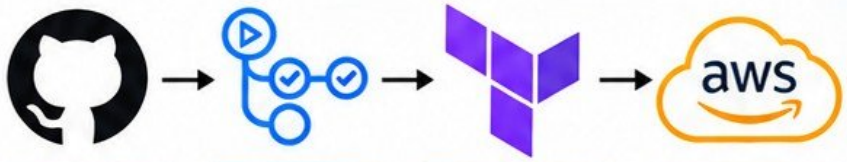
Production Lesson — A stable serverless API is more than a working Lambda—it needs correct routes, permissions, and verified integrations.

Provision AWS Infrastructure Using Terraform

What You Will Do

- Store Terraform configuration in GitHub.
- Configure a remote S3 backend with state locking.
- Run terraform fmt, validate, and plan on pull requests.
- Apply approved changes through GitHub Actions.
- Use OIDC and separate planning and deployment permissions.

Architecture: Terraform + AWS + GitHub



GitHub Repository (Terraform Code) → GitHub Actions (Plan / Apply) → Terraform (Infrastructure as Code) → AWS (Provisioned Resources)

Key Concepts

Terraform	Infrastructure as Code (IaC) to create, update, and manage AWS resources.
S3 Backend	Stores the Terraform state file remotely.
State Locking	Prevents concurrent changes (e.g., DynamoDB lock table).
OIDC	Secure, short-lived AWS access from GitHub Actions (no access keys).

Deployment Flow

1. Push Terraform code to GitHub (create a pull request).
2. Run terraform fmt, validate, and plan via GitHub Actions.
3. Review the plan output in the pull request.
4. After approval, run terraform apply via GitHub Actions.
5. Provision or update AWS resources (e.g., VPC, EC2, S3, ALB, ECS, etc.).

GitHub Actions (Workflow Snippet)

```yaml
name: Terraform (Plan & Apply)
on:
  pull_request:
    branches: [ main ]
  workflow_dispatch:
jobs:
  terraform-plan:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform fmt -check
      - run: terraform validate
      - run: terraform plan -out=tfplan
  terraform-apply:
    needs: terraform-plan
    if: github.event_name == 'workflow_dispatch'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform apply "tfplan"
```

terraform Commands

terraform init	Initialize the working directory and configure the backend (S3).
terraform fmt	Format configuration files consistently.
terraform validate	Check for syntax and configuration errors.
terraform plan	Show the proposed changes (create, update, delete).
terraform apply	Apply the approved changes to AWS.
terraform destroy	Remove the provisioned resources (use with caution).

Common Issues & Fixes

Issue	Likely Cause	Fix
State lock error	Previous run did not release the lock.	Verify the DynamoDB lock table and clear stale locks.
AccessDenied	OIDC role missing required permissions.	Update the IAM role policy (e.g., S3, EC2, VPC, etc.).
Plan shows unexpected changes	Configuration drift or manual changes in AWS.	Review the plan and align resources with code (terraform import if needed).
Backend init fails	Incorrect S3 bucket, region, or permissions.	Confirm the S3 backend config and IAM access.

Production Warning Never automatically apply unreviewed infrastructure changes. Always review and approve the plan.

Deploy AWS CloudFormation Stacks

What You Will Do

- Create infrastructure using CloudFormation templates.
- Validate templates before deployment.
- Deploy changes using GitHub Actions.
- Inspect CloudFormation change sets and stack events.
- Handle failed updates and rollback states.
- Understand when teams choose CloudFormation instead of Terraform.

Deployment Architecture



GitHub Repository (Template) → GitHub Actions (Validate & Deploy) → AWS CloudFormation (Stack) → AWS Resources (VPC, EC2, S3, etc.)

Key Concepts

CloudFormation	AWS native IaC service that provisions and manages resources.
Template	YAML or JSON file that defines AWS resources.
Change Set	A preview of changes before applying to a stack.
Rollback	Reverts changes when a deployment fails.
Stack Event	A detailed log of what happened during deployment.

Deployment Steps

1. Write a CloudFormation template (e.g., template.yaml).
2. Validate the template syntax and resources (e.g., cfn-lint or aws cloudformation validate-template).
3. Create a GitHub Actions workflow for validate and deploy.
4. Create a change set and review the proposed changes.
5. Deploy the stack and monitor stack events.
6. Handle failed updates and rollback if needed.

GitHub Actions (Workflow Snippet)

```yaml
name: Deploy CloudFormation Stack
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install AWS CLI
        run: sudo apt-get install -y awscli
      - name: Validate template
        run: aws cloudformation validate-template \
             --template-body file://template.yaml
      - name: Create change set
        run: aws cloudformation create-change-set \
             --stack-name veriqta-app \
             --template-body file://template.yaml \
             --change-set-name cf-${{ github.sha }}
      - name: Deploy stack
        run: aws cloudformation execute-change-set \
             --stack-name veriqta-app \
             --change-set-name cf-${{ github.sha }}
```

Common Commands

`aws cloudformation validate-template`	Validate a template's syntax.
`aws cloudformation create-change-set`	Create a change set for proposed changes.
`aws cloudformation execute-change-set`	Apply the approved change set.
`aws cloudformation describe-stacks`	View stack status and outputs.
`aws cloudformation describe-stack-events`	See detailed deployment events.

Common Issues & Fixes

Issue	Likely Cause	Fix
CREATE_FAILED / UPDATE_FAILED	Invalid resource configuration or missing permissions.	Check stack events, fix the template, and redeploy.
ROLLBACK_IN_PROGRESS / ROLLBACK_COMPLETE	Deployment failed and changes are being reverted.	Review the failure reason, update the template, and create a new change set.
Template validation error	Syntax error or unsupported resource properties.	Run validate-template (or cfn-lint) and correct the template.

When to Choose CloudFormation

- You prefer an AWS native solution (fully integrated with AWS).
- You want tight integration with other AWS services (e.g., StackSets, Service Catalog).
- You work in environments where CloudFormation is the standard.

Production Takeaway

Use change sets, review the impact, and monitor stack events. Always have a rollback plan.

Deploy Kubernetes Applications to Amazon EKS

What You Will Do

- Authenticate GitHub Actions to an EKS cluster.
- Configure AWS IAM permissions and Kubernetes RBAC access.
- Build and publish a container image to Amazon ECR.
- Update a Kubernetes Deployment using an immutable image reference.
- Verify Pods, Services, and rollout status.
- Troubleshoot ImagePullBackOff, CrashLoopBackOff, and readiness probe failures.

Architecture: GitHub Actions → Amazon EKS



GitHub Repository (App Code) → GitHub Actions (Build, Push & Deploy) → Amazon ECR (Container Image) → Amazon EKS (Kubernetes Cluster) → Kubernetes (Deployment, Service, Pods)

Key Concepts

Amazon EKS	Managed Kubernetes service by AWS.
Amazon ECR	Stores container images securely.
Kubernetes Deployment	Runs and manages replicated application Pods.
IAM & RBAC	Controls who can access AWS and Kubernetes resources.

Deployment Steps

1. Configure IAM permissions and EKS access (for GitHub Actions).
2. Build and push the container image to ECR (with a unique tag, e.g., commit SHA).
3. Update the Kubernetes manifests (e.g., Deployment) with the new immutable image tag.
4. Deploy to EKS using kubectl or a workflow action (e.g., aws-actions/eks-deploy).
5. Verify Pods, Services, and rollout status.
6. Troubleshoot issues (ImagePullBackOff, CrashLoopBackOff, readiness probe failures).

GitHub Actions (Workflow Snippet)

```yaml
name: Deploy to Amazon EKS
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
      - uses: aws-actions/amazon-ecr-login@v2
      - name: Build and push image
        run: |
          docker build -t veriqta-api:${{ github.sha }} .
          docker push $ECR_URI:${{ github.sha }}
      - name: Update and deploy to EKS
        run: |
          kubectl apply -f k8s/deployment.yaml
          kubectl rollout status deployment/veriqta-api
```

Verify the Deployment

`kubectl get pods`	List Pods and their status (Running, Pending, etc.).
`kubectl get svc`	View Services and cluster endpoints.
`kubectl rollout status deployment/veriqta-api`	Check if the deployment completed successfully.
`kubectl logs <pod-name>`	View application logs for errors.
`kubectl describe pod <pod>`	Get detailed information and failure reasons.

Common Failures & Fixes

Issue	Likely Cause	Fix
ImagePullBackOff	Wrong image tag, ECR permission, or image not found.	Verify ECR image, tag, and IAM permissions.
CrashLoopBackOff	Application crashes on startup (code, config, or environment).	Check Pod logs, fix the app error, and redeploy.
Readiness probe failed	Health endpoint not responding or wrong port/path.	Verify probe path, port, and app health endpoint.

✓ Production Takeaway

A successful EKS deployment is not just about deploying the manifest — it's about making the application run, stay healthy, and be reachable.

Release Applications to EKS Using Helm

What You Will Do

- Package a Kubernetes application as a Helm chart.
- Separate configuration using values files.
- Validate and lint charts before deployment.
- Use helm upgrade --install with --atomic and --wait.
- Inspect release history and perform rollback.
- Verify the application health after the release.

Architecture: Helm + EKS (GitHub Actions)



GitHub Repository (Helm Chart) → GitHub Actions (Lint, Package & Deploy) → Helm (Chart Release) → Amazon EKS (Kuberentes Cluster) → Kubernetes (Deployment, Service, Pods)

Key Concepts

Helm	A package manager for Kubernetes.
Helm Chart	A reusable package with Kubernetes manifests (templates + values).
Values File	A configuration file (e.g., values.yaml) used to customize a release.
Helm Upgrade	Updates or installs a release in the cluster.
Rollback	Reverts to a previous successful release.

Deployment Steps

1. Create a Helm chart for the application (helm create or from a template).
2. Configure environment-specific values (e.g., values-dev.yaml, values-prod.yaml).
3. Validate and lint the chart (helm lint and helm template).
4. Deploy or upgrade the release: helm upgrade --install <release> ./chart \ --atomic --wait --values values.yaml.
5. Check the release history (helm history <release>).
6. Roll back if needed (helm rollback <release> <revision>).

Helm Commands (Quick Reference)

`helm create <name>`	Create a new Helm chart.
`helm lint ./chart`	Validate the chart for issues.
`helm template ./chart`	Render manifests without deploying.
`helm upgrade --install <release> ./chart`	Install or upgrade a release.
`--atomic --wait`	Roll back on failure and wait for resources to be ready.
`helm history <release>`	View release history and revisions.
`helm rollback <release> <n>`	Revert to a previous revision.

GitHub Actions (Workflow Snippet)

```yaml
name: Deploy to EKS with Helm
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Helm
        uses: azure/setup-helm@v4
      - name: Lint Chart
        run: helm lint ./chart
      - name: Deploy Release
        run: |
          helm upgrade --install veriqta-api ./chart \
          --atomic --wait --values values.yaml
```

Common Failures & Fixes

Issue	Likely Cause	Fix
Release failed (rolled back)	Manifest error, resource conflict, or health check failure.	Check helm status, inspect logs, and fix the issue.
Pods not ready	Wrong image, bad configuration, or failed readiness probe.	Verify image, env vars, and readinessProbe/livenessProbe.
Chart lint error	Syntax or template error (e.g., incorrect values or format).	Run helm template and fix the reported error.
Upgrade succeeded but app not working	Application misconfiguration or dependency issue.	Check pod logs, events, and service endpoints.

Verify the Release

- Check release status: `helm status veriqta-api`
- Check Pods: `kubectl get pods -l app=veriqta-api`
- Check Services: `kubectl get svc veriqta-api`
- Check rollout: `kubectl rollout status deploy/veriqta-api`

Production Lesson

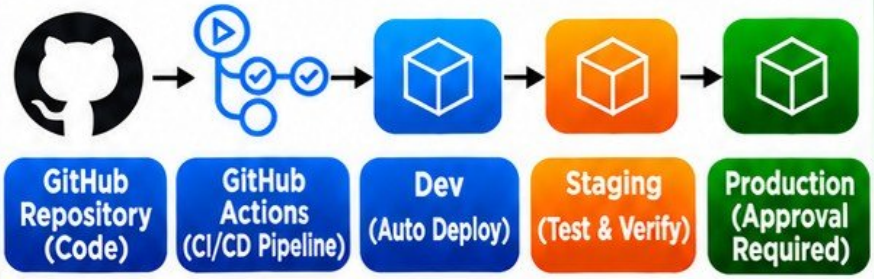
A Helm release needs health verification, not just a successful command.

Build Separate Development, Staging, and Production Pipelines

What You Will Do

- [x] Create GitHub environments with deployment protection rules.
- [x] Map each environment to its own AWS role and resources.
- [x] Deploy automatically to development.
- [x] Require approval before production deployment.
- [x] Promote the same tested artifact (image) across environments.

Pipeline Flow: Dev → Staging → Production



Key Components

GitHub Environments	Dev, Staging, Production with rules and approvals.
Protection Rules	Control who can deploy, require reviews, and add wait timers.
AWS IAM Role	A separate role per environment with least privilege.
Artifact (Image)	A single, tested container image promoted across environments.
CI/CD Pipeline	Build once, test, and deploy to each environment.

Deployment Steps

1. Create three GitHub environments (dev, staging, production).
2. Configure protection rules (e.g., required reviewers for production).
3. Create AWS IAM roles and map each role to its environment.
4. Build and push the application image to ECR with a unique tag (e.g., commit SHA).
5. Deploy automatically to the development environment.
6. Deploy to staging and run tests (e.g., smoke tests).
7. Promote the same tested image to production with approval.

GitHub Actions (Example Workflow)

```yaml
name: Multi-Environment Deployment
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build and push image to ECR
        run: |
          docker build -t veriqta-api:${{ github.sha }} .
          docker push $ECR_URI:${{ github.sha }}
  deploy-dev:
    environment: development
    needs: build
  deploy-staging:
    environment: staging
    needs: deploy-dev
  deploy-prod:
    environment: production
    needs: deploy-staging
```

Environment Mapping (Example)

Environment	AWS Role	Resources (Example)
Development	arn:aws:iam::1111:role/dev-deploy	ECR (dev), EKS (dev)
Staging	arn:aws:iam::1111:role/staging-deploy	ECR (staging), EKS (staging)
Production	arn:aws:iam::1111:role/prod-deploy	ECR (prod), EKS (prod)

Common Failure & Fixes

Issue	Likely Cause	Fix
Prod deployed without testing	Pipeline bypassed staging or approval.	Enforce environment protection rules and required reviewers.
Wrong version deployed	Image rebuilt instead of promoted.	Promote the same image tag (e.g., commit SHA) across environments.
Access denied (403)	IAM role lacks permissions.	Update the IAM role policy for the target environment.

Common Failure Deploying an untested image to production.

Secure CI/CD with Dependency and Container Scanning

What You Will Do

- Scan application dependencies before building.
- Scan container images for known vulnerabilities.
- Configure GitHub Actions permissions using least privilege.
- Pin third-party actions to reviewed commit SHAs.
- Block releases when security thresholds fail.
- Handle false positives and vulnerability exceptions responsibly.

Pipeline Flow: Build → Scan → Gate → Deploy



GitHub Push (Code) → Build (App + Image) → Scan (Deps & Image) → Gate (Block if High Risk) → Container (ECR) → Deploy (EKS/ECS)

Key Concepts

Dependency Scanning	Checks project libraries (e.g., npm, pip) for known vulnerabilities.
Container Scanning	Scans container images for security issues (CVEs, misconfigurations).
Least Privilege	Give GitHub Actions only the minimal permissions it needs.
Pin Actions (Commit SHA)	Use a specific commit SHA (e.g., actions/checkout@<sha>) instead of a moving tag.
Security Gate	Block the release when vulnerabilities exceed defined thresholds.

Deployment Steps

1. Add dependency scanning (e.g., Trivy, OWASP Dependency-Check, or npm audit).
2. Build the container image.
3. Scan the image for vulnerabilities (e.g., Trivy or Grype).
4. Configure least-privilege permissions in GitHub Actions (OIDC to AWS).
5. Pin all third-party actions to a specific commit SHA.
6. Set security thresholds (e.g., fail on High or Critical).
7. Handle false positives and manage approved exceptions (with a justification).

GitHub Actions (Workflow Snippet)

```yaml
name: Build, Scan and Deploy
on:
  push:
    branches: [ main ]
permissions:
  contents: read
  id-token: write  # for AWS OIDC
jobs:
  build-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@
        a3e5f01d7b...    # pinned SHA
      - name: Scan dependencies
        run: trivy fs --severity HIGH,CRITICAL .
      - name: Build image
        run: docker build -t veriqta-api:${{ github.sha }} .
      - name: Scan image
        run: trivy image --severity HIGH,CRITICAL .
             veriqta-api:${{ github.sha }}
      - name: Fail on high vulnerabilities
        run: trivy image --exit-code 1
          --severity HIGH,CRITICAL veriqta-api:${{ github.sha }}
```

Security Scanning Tools (Examples)

Trivy	Lightweight scanning for dependencies and container images.
Grype	Fast container image vulnerability scanner.
OWASP Dependency-Check	In-depth scan for application dependencies (works in CI/CD).

Common Issues & Fixes

Issue	Likely Cause	Fix
Build blocked by vulnerability	High/Critical issue exceeds threshold.	Fix or upgrade the affected dependency, then re-run the scan.
False positive	Scanner flags a low-risk or non-applicable issue.	Review, document the reason, and add a controlled exception.
Permission denied (403)	GitHub Actions role lacks required AWS permissions.	Update the IAM role policy with only the needed actions (e.g., ECR, ECS, EKS).

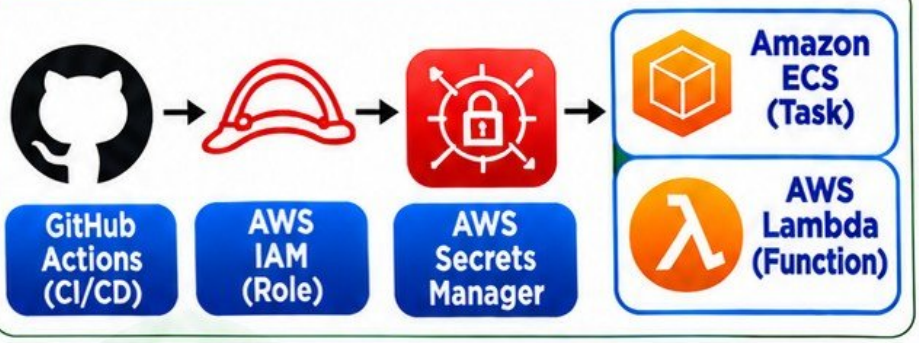
Production Takeaway — A secure pipeline scans early, uses least privilege, and only deploys when the risk is acceptable.

Manage AWS Secrets Without Exposing Credentials

What You Will Do

- Store application secrets in AWS Secrets Manager.
- Use IAM roles to authorize application access.
- Keep sensitive values out of GitHub workflow logs and artifacts.
- Configure ECS or Lambda to retrieve secrets securely.
- Test access and diagnose AccessDeniedException.

Architecture: GitHub → AWS → Secrets Manager



Key Concepts

AWS Secrets Manager	A secure, managed service to store and retrieve secrets.
IAM Role	Grants applications permission to read secrets (least privilege).
ECS / Lambda	Application runtimes that can securely fetch secrets at runtime.
Least Privilege	Give only the required secrets permission (e.g., secretsmanager:GetSecretValue).
Secret Hygiene	Keep secrets out of code, GitHub logs, and build artifacts.

Implementation Steps

1. Create a secret in AWS Secrets Manager (e.g., /veriqta/app/db).
2. Create an IAM role with permission: secretsmanager:GetSecretValue.
3. Attach the role to ECS (task role) or Lambda (execution role).
4. Update the application to retrieve the secret at runtime (using AWS SDK or environment configuration).
5. Deploy and test the application (ECS or Lambda).
6. If access fails, check the IAM policy, role attachment, secret ARN, and region.

Example: Create and Fetch a Secret

```
# Create a secret in AWS Secrets Manager
aws secretsmanager create-secret \
  --name /veriqta/app/db \
  --secret-string '{"username":"appuser",
 "password":"StrOngP@ss!"}'
```

```
# Fetch the secret in Python (app code)
import boto3, json
client = boto3.client("secretsmanager")
resp = client.get_secret_value(
   SecretId="/veriqta/app/db"
)
secret = json.loads(resp["SecretString"])
db_user = secret["username"]
db_pass = secret["password"]
```

Common Issues & Fixes

Issue	Likely Cause	Fix
AccessDenied Exception	IAM role missing secretsmanager: GetSecretValue permission.	Update the IAM policy and ensure the role is attached to ECS/Lambda.
Secret not found	Wrong secret name or ARN (or wrong region).	Verify the secret name/ARN and the AWS region.
Secret value exposed in logs	Secret printed in code or GitHub workflow logs.	Remove secret output and mask sensitive values (e.g., use GitHub secrets and avoid echoing values).
Application cannot read secret	Incorrect role (task vs. execution role) or policy.	Use the correct role (ECS task role or Lambda execution role) with the right permissions.

Deployment vs. Application Secrets

Deployment Credentials	Used by GitHub Actions to deploy (e.g., via OIDC or a role for CI/CD).
Application Secrets	Used by the running application (e.g., DB password, API keys) and stored in AWS Secrets Manager.

✓ Production Lesson

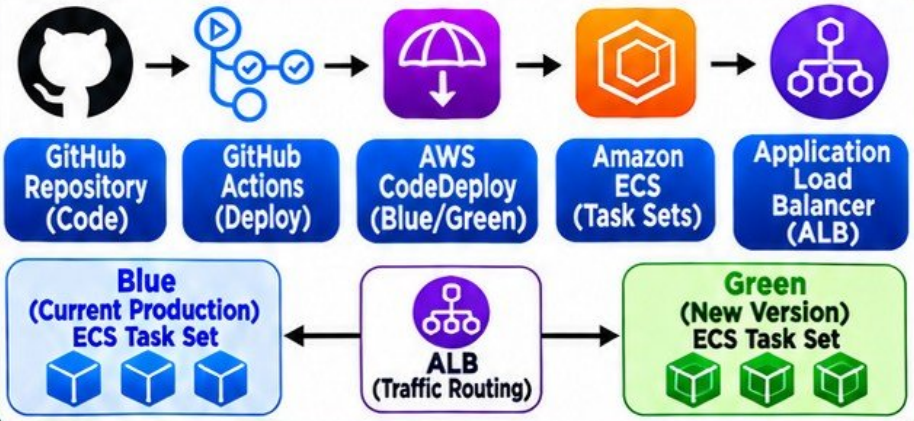
Deployment credentials and application secrets serve different purposes.

Implement Blue/Green Deployment with Amazon ECS

What You Will Do

- Create blue and green ECS task sets.
- Integrate CodeDeploy with an Application Load Balancer (ALB).
- Deploy a new version through GitHub Actions.
- Validate the new environment before switching traffic.
- Configure automatic rollback using CloudWatch alarms.

Architecture: Blue/Green Deployment on ECS



Key Concepts

Amazon ECS	Runs and manages containerized applications using task sets.
AWS CodeDeploy	Orchestrates blue/green deployments and traffic shifting.
Application Load Balancer (ALB)	Routes traffic to blue or green task sets.
Task Set	A group of ECS tasks (blue or green version).
CloudWatch Alarm	Monitors health and triggers automatic rollback on failure.

Deployment Steps

1. Create an ECS service with two task sets (blue and green).
2. Configure CodeDeploy for ECS (blue/green) with the ALB and target groups.
3. Build and push the new container image (e.g., to Amazon ECR) via GitHub Actions.
4. Deploy the new version through CodeDeploy to the green task set.
5. Validate the green environment (health checks, smoke tests, or key application checks).
6. Shift production traffic to green.
7. Configure CloudWatch alarms and enable automatic rollback to blue if the new version fails.

GitHub Actions (Workflow Snippet)

```yaml
name: Deploy to ECS (Blue/Green)
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build and push image to ECR
        run: docker build -t veriqta-app:${{ github.sha }} .
             docker push $ECR_URI:${{ github.sha }}
      - name: Deploy with CodeDeploy
        run: |
          aws deploy create-deployment \
            --application-name veriqta-ecs-app \
            --deployment-group-name ecs-bluegreen \
            --revision revisionType=AppSpec,appSpecContent={
              imageUri=$ECR_URI:${{ github.sha }}
```

Common Issues & Fixes

Issue	Likely Cause	Fix
New tasks healthy but app fails	Application-level error (e.g., config, DB, or runtime issue).	Validate app health (endpoints, logs, and metrics) before traffic shift.
Traffic not shifting	Incorrect target group or CodeDeploy config.	Verify ALB target groups, listener rules, and deployment group settings.
Automatic rollback not triggered	Missing or misconfigured CloudWatch alarm.	Create a CloudWatch alarm (e.g., high 5xx error rate) and link it to CodeDeploy.
Deployment stuck	Health check failure or tasks not reaching steady state.	Check ECS task logs, health checks, and resource limits (CPU, memory).

Real-World Failure New tasks can start successfully but still fail application-level checks (e.g., login, API, or business logic).

Automated Rollback After a Failed Production Release

What You Will Do

- Detect release failures using health checks and CloudWatch alarms.
- Understand the difference between pipeline failures and runtime failures.
- Roll back to a previously verified (stable) deployment.
- Capture logs and deployment metadata for investigation.
- Verify that the rolled-back version is healthy and serving traffic.

Rollback Flow: Detect → Rollback → Verify



CloudWatch Alarm (Detect) → Release Failure (Alert) → Automatic Rollback (Previous Version) → Service Healthy (Verify)

Key Concepts

CloudWatch Alarm	Monitors key metrics (e.g., 5xx errors, high latency, unhealthy targets).
Health Check	Confirms the application is running and working (e.g., /health endpoint).
Rollback	Revert traffic to a previous, verified (healthy) version.
Pipeline Failure	A CI/CD failure (e.g., build, test, or deploy step fails).
Runtime Failure	The deployment succeeds, but the application fails after release (e.g., errors, crashes, high 5xx).

Implementation Steps

1. Define health checks for the application (e.g., /health or /ready).
2. Create CloudWatch alarms (e.g., high 5xx rate, high latency, or unhealthy targets).
3. Configure automatic rollback (e.g., CodeDeploy or ECS deployment with alarms).
4. Distinguish pipeline failures (build/test/deploy) from runtime failures (post-deploy issues).
5. Roll back to the last known good version (e.g., CodeDeploy, ECS, or Helm).
6. Collect logs and deployment metadata (e.g., GitHub run logs, ECS/ALB logs, CloudWatch).
7. Verify the previous version is healthy (endpoints, metrics, and key functionality).

Failure Types: Pipeline vs. Runtime

Pipeline Failure (Pre-Release)	Runtime Failure (Post-Release)
Happens during build, test, or deploy.	Happens after a successful deployment.
Example: build failed, test failed, or deploy error.	Example: app crashes, 5xx errors, or health check fails.
Fix: correct the pipeline issue and redeploy.	Fix: roll back to the previous healthy version and investigate.

Rollback Example (GitHub Actions)

```yaml
name: Deploy with Rollback
on:
  workflow_dispatch:
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
    - name: Deploy to production
      run: aws deploy create-deployment \
        --application-name veriqta-app \
        --deployment-group-name prod \
        --revision revisionType=GitHub,\\
          gitHubLocation={repository=veriqta/app,\
          commitId=${{ github.sha }} }
```

Common Issues & Fixes

Issue	Likely Cause	Fix
Rollback not triggered	Alarm not configured or threshold too high.	Set correct alarms and rollback in CodeDeploy/ECS.
Rolled back but still unhealthy	Previous version has hidden issues or missing config.	Verify the previous version fully (health, logs, metrics).
Hard to investigate	Logs and metadata not captured.	Collect GitHub, ECS/ALB, and CloudWatch logs for analysis.

Production Lesson

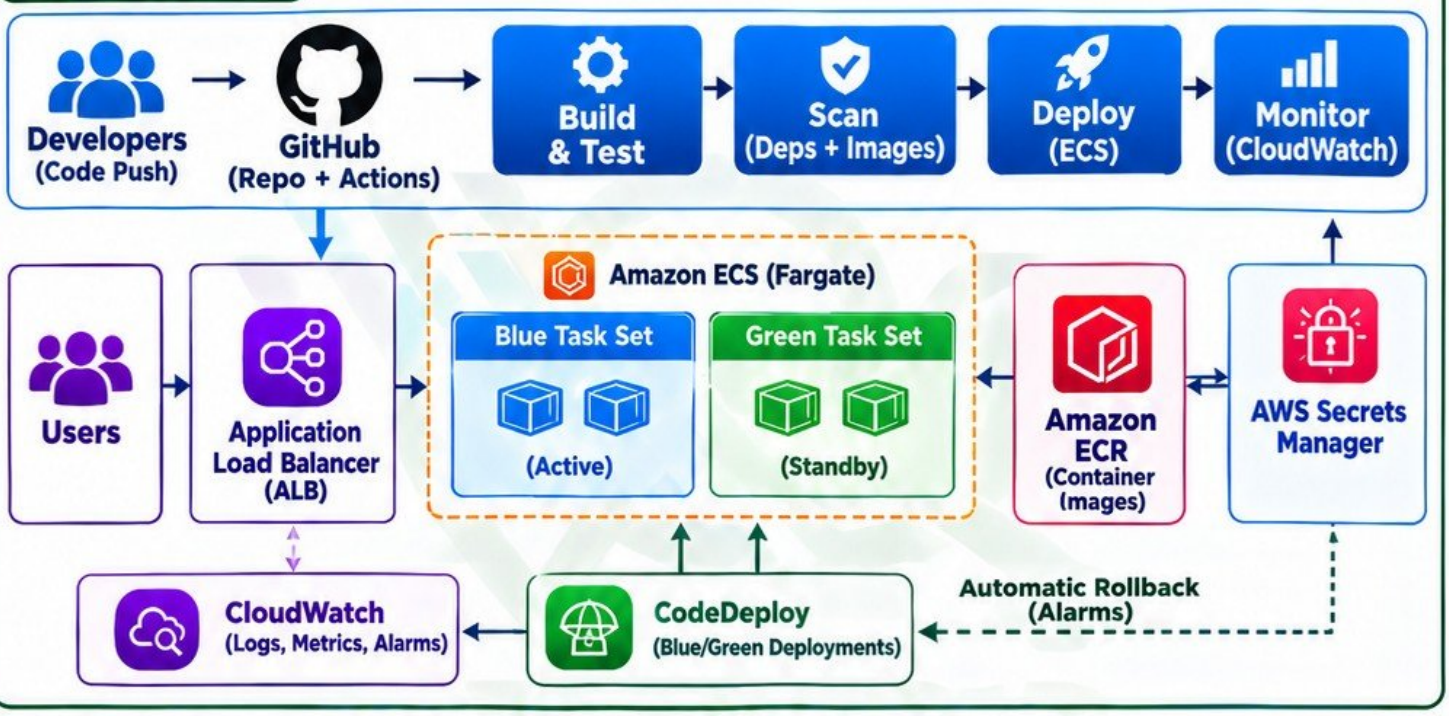
Rollback is not complete until service recovery is confirmed.

Final Production Challenge: Build a Complete AWS CI/CD Pipeline

Scenario: Deploy a Production-Ready Customer API

Your engineering team owns a containerized API running on Amazon ECS Fargate. Developers need a secure, automated deployment process that can detect failures and recover safely.

Architecture



Key Implementation Steps

1. Build a Docker image and push to ECR (automated via GitHub Actions).
2. Create two ECS task sets (blue and green) with the same task definition.
3. Configure an Application Load Balancer with target groups for both task sets.
4. Deploy using CodeDeploy (blue/green).
5. Set up CloudWatch alarms for health checks and automatic rollback.
6. Store and manage secrets with AWS Secrets Manager.

Key Requirements

Security	Use IAM roles, Secrets Manager, least privilege, and encrypted connections.
Automation	CI/CD with GitHub Actions and CodeDeploy.
Monitoring	CloudWatch logs, metrics, and alarms.
Resilience	Blue/green deployment with automatic rollback.
Scalability	ECS Fargate for serverless containers.

Production Lesson

A complete deployment is not successful until the application is healthy and serving traffic.